You said:

Act as my programming tutor. Keep responses concise and beginner-friendly. Give me one coding task at a time, let me implement it, and then review my code. Avoid unnecessary explanations, complete replacement files and additional features. My goal is to write and understand most of the implementation myself.

Memory updated

ChatGPT said:

Absolutely. I'll act as your programming tutor and follow these rules:

1. **One task at a time.** I'll give you a single, manageable coding task.
2. **You write the code.** I'll let you implement it rather than doing it for you.
3. **I'll review your code.** I'll identify errors, explain why they happen, and suggest improvements.
4. **Beginner-friendly explanations.** I'll explain concepts in straightforward language without overwhelming you.
5. **Hints before solutions.** If you're stuck, I'll offer hints before showing example code.
6. **No unnecessary extras.** I won't introduce additional features, rewrite entire files, or overcomplicate tasks.

We'll follow this cycle:

1. **I give you a coding task**
2. **You implement it**
3. **I review your code**
4. **We move to the next task**

What language or project would you like to work on first?

You said:

I am developing crash_risk_rules.pl for my COMP3742/9742 hybrid AI traffic-risk assessment project.

Python handles machine learning and generates intersection facts. SWI-Prolog uses these facts to assess data quality, traffic exposure and pedestrian or school crossing conditions.

Help me plan the Prolog knowledge base and explain what I need to implement. Keep explanations brief and let me write the code myself.

ChatGPT said:

Good. We'll approach `crash_risk_rules.pl` as a small, explainable **Prolog knowledge base**, rather than trying to put all the project's intelligence into one large set of rules.

Your Python component will generate the intersection facts, while Prolog will reason over those facts and produce infrastructure-risk assessments.

I'll help you design it incrementally, and you'll write the implementation.

1. Proposed knowledge base structure

I'd organise `crash_risk_rules.pl` into five logical sections.

Section	Responsibility
1. Data facts	Receive intersection data from Python
2. Data quality	Identify missing or unreliable information
3. Traffic exposure	Assess traffic volume and exposure
4. Infrastructure	Assess pedestrian and school crossing conditions
5. Risk assessment	Combine evidence into explainable priorities

Section 1 – Data facts

These are supplied by Python, not manually calculated by Prolog.

Proposed predicate signatures:

```
intersection(IntersectionID).
predicted_risk(IntersectionID, RiskScore).
traffic_volume(IntersectionID, VehiclesPerDay).
pedestrian_crossing(IntersectionID).
school_crossing(IntersectionID).
traffic_signal(IntersectionID).
```

These are examples of a possible interface. We should match them to your actual Python output before implementing them.

Section 2 – Data quality

Prolog should identify whether an intersection has sufficient information for assessment.

Potential rules:

- Required information is missing.
- Traffic volume is unavailable.
- Infrastructure information is incomplete.
- An intersection has sufficient data to proceed.

Importantly, missing data should not automatically mean that infrastructure is absent.

Section 3 – Traffic exposure

Assess traffic exposure using traffic-volume information.

Potential classifications:

- Low traffic exposure.
- Moderate traffic exposure.
- High traffic exposure.

We'll need to establish appropriate thresholds from your dataset rather than inventing arbitrary values.

Section 4 – Pedestrian and school crossing conditions

Assess infrastructure availability.

For example:

- No pedestrian crossing recorded.
- Pedestrian crossing present.
- School crossing present.
- Both crossing types present.

These conditions can contribute to infrastructure-priority decisions.

Section 5 – Risk assessment

This is where the hybrid AI approach becomes useful.

Prolog combines the machine-learning prediction with traffic exposure and infrastructure conditions.

For example:

Evidence	Potential interpretation
High predicted risk	Increased concern
High traffic exposure	More road-user exposure
No pedestrian crossing	Potential infrastructure gap
School crossing present	Existing protective infrastructure

The final output should explain *why* an intersection receives a particular priority.

2. Suggested implementation order

We'll work through these tasks sequentially.

1. **Define the input predicates**

   Establish the facts Python supplies to Prolog.

2. **Implement data-quality rules**

   Identify missing information and determine assessment eligibility.

3. **Implement traffic-exposure rules**

   Classify intersections using traffic-volume thresholds.

4. **Implement infrastructure rules**

   Evaluate pedestrian and school crossing conditions.

5. **Implement risk-priority rules**

   Combine the evidence into explainable recommendations.

6 **Test the knowledge base**

Test normal cases, missing data and conflicting conditions.

3. Your first coding task

Task 1 – Input predicates

Before writing any reasoning rules, we need to establish how Python communicates with Prolog.

Your task:

Create `crash_risk_rules.pl` and define the input predicate structure for:

1. Intersection identification.
2. Predicted crash risk.
3. Traffic volume.
4. Pedestrian crossing availability.
5. School crossing availability.
6. Traffic signal availability.

For now, only define the predicates or add a few example facts to test their structure. Don't implement any reasoning rules yet.

One question before you start: Does your Python pipeline already generate a specific set of Prolog facts, or are we designing the Python-to-Prolog data format together?

Once you've answered and written your first attempt, paste your code here. I'll review it without replacing your implementation.

You said:

Help me implement the data-validation and availability predicates in crash_risk_rules.pl.

The system needs to identify missing traffic-volume information, stale data and incomplete intersection information.

Give me one small coding task at a time. Explain how to test each predicate and review my implementation before moving on.

ChatGPT said:

We'll build the validation section incrementally. We'll use **SWI-Prolog**, and I'll review each predicate before giving you the next task.

We'll separate three concepts:

1. **Missing data** – required information does not exist.
2. **Stale data** – information exists but is too old.
3. **Incomplete data** – an intersection is missing one or more required attributes.

One important distinction: missing information should be reported as a data-quality issue, not automatically interpreted as missing infrastructure.

Task 1 – Detect missing traffic-volume information

Goal: Create a predicate that identifies intersections without a traffic-volume fact.

Assume your Python-generated facts look something like:

```
intersection(int_001).
traffic_volume(int_001, 12500).
intersection(int_002).
```

Here, `int_002` has no traffic-volume information.

Your task: Implement this predicate:

```
missing_traffic_volume(IntersectionID).
```

It should succeed when:

- The intersection exists.
- No traffic-volume fact exists for that intersection.

How to test it

Once implemented, open SWI-Prolog and load your file:

```
?- [crash_risk_rules].
```

Then test:

```
?- missing_traffic_volume(int_001).
?- missing_traffic_volume(int_002).
```

Expected behaviour:

Query	Expected result
`int_001`	`false`
`int_002`	`true`

You can also test whether Prolog finds all intersections with missing traffic-volume information.

A small hint: Prolog's negation operator is `\+`.

Try implementing the predicate yourself. Paste your code here, along with any errors or test results.

I'll review it before we move on to detecting stale data.

You said:

Continue with task 2

ChatGPT said:

Task 2 – Detect stale traffic-volume data

Next coding task

Goal: Identify intersections where traffic-volume information exists but is too old to be considered reliable.

1. Understand the input facts

We'll assume Python supplies traffic-volume facts with an associated collection year:

```
traffic_volume(int_001, 12500).
traffic_volume_year(int_001, 2020).

traffic_volume(int_002, 8500).
traffic_volume_year(int_002, 2025).
```

For this exercise, assume the current assessment year is **2026**, and data older than 3 years is considered stale.

2. Your coding task

Implement this predicate:

```
stale_traffic_volume(IntersectionID).
```

It should succeed when:

- The intersection has a traffic-volume year.
- That year is more than 3 years before the assessment year.

Use `2026` as the assessment year for now.

Hint: Prolog arithmetic comparisons use `>`, and you can subtract years using `is`.

3. Test your predicate

Expected results:

Intersection	Data year	Expected
`int_001`	2020	`true`
`int_002`	2025	`false`

Try these queries:

```
?- stale_traffic_volume(int_001).
?- stale_traffic_volume(int_002).
```

Your turn

Write the `stale_traffic_volume/1` predicate in your existing file and paste your implementation here.

I'll check:

- Whether your arithmetic is correct.
- Whether missing year information is handled safely.
- Whether the predicate behaves correctly for both test cases.

We won't move to Task 3 until we've reviewed your implementation.

You said:

Move on to task 3

ChatGPT said:

Task 3 – Detect incomplete intersection information

Next coding task

Goal: Identify intersections that are missing required information.

We'll check whether each intersection has:

1. A predicted risk score.
2. Traffic-volume information.
3. Traffic-volume collection year.

1. Example input facts

```
intersection(int_001).
predicted_risk(int_001, 0.82).
traffic_volume(int_001, 12500).
traffic_volume_year(int_001, 2025).

intersection(int_002).
predicted_risk(int_002, 0.65).
```

Here, `int_002` is incomplete because it lacks traffic-volume information and its collection year.

2. Your coding task

Implement:

```
incomplete_intersection(IntersectionID).
```

The predicate should succeed if an existing intersection is missing at least one required piece of information.

Hints:

- Use `intersection/1` to identify valid intersections.
- Use `\+` to check whether information is missing.
- Use `;` for logical OR.
- You can reuse your existing `missing_traffic_volume/1` predicate.

3. Test your implementation

Try:

```
?- incomplete_intersection(int_001).
?- incomplete_intersection(int_002).
```

Expected results:

Intersection	Expected
`int_001`	`false`
`int_002`	`true`

Also test an intersection that has traffic volume but is missing its predicted risk score.

Your turn

Write `incomplete_intersection/1` yourself and paste your code here.

I'll review your logic, including whether it correctly identifies multiple missing fields, before we move to the next task.

You said:

Help me implement the contextual reasoning rules in crash_risk_rules.pl.
The system should consider:

- Traffic volume, using the existing 20,000-vehicle threshold.
- Pedestrian crossings.
- School crossings.

Guide me through implementing these conditions and combining them using Prolog rules. Let me write the code.

ChatGPT said:

We'll build the contextual reasoning section one predicate at a time. The goal is to let Prolog identify relevant road-safety conditions and eventually combine them into infrastructure-priority recommendations.

We'll reuse your existing Python-generated facts rather than duplicating data.

Planned implementation

We'll work through four small tasks:

1. **High traffic exposure** – identify intersections with traffic volume above 20,000 vehicles.
2. **Pedestrian crossing conditions** – identify intersections with and without pedestrian crossings.
3. **School crossing conditions** – identify intersections with and without school crossings.
4. **Combined contextual reasoning** – combine traffic exposure and crossing conditions into a higher-priority assessment.

We won't implement all four at once.

Task 1 – Identify high traffic exposure

Your current task

Objective: Create a Prolog rule that identifies intersections with high traffic exposure.

Your existing traffic-volume facts should look something like:

```
traffic_volume(int_001, 25000).
traffic_volume(int_002, 15000).
traffic_volume(int_003, 32000).
```

The existing threshold is **20,000 vehicles**.

What you need to implement

Create a predicate with this signature:

```
high_traffic_exposure(IntersectionID).
```

It should succeed when:

- Traffic-volume information exists for the intersection.
- The recorded traffic volume is greater than 20,000.

Hints

You'll need:

- `traffic_volume/2` to retrieve the volume.
- A Prolog arithmetic comparison operator.
- A rule using `:-`.

Remember that Prolog uses `>` for greater-than comparisons.

Testing

After implementing your predicate, try:

```
?- high_traffic_exposure(int_001).
?- high_traffic_exposure(int_002).
?- high_traffic_exposure(int_003).
```

Expected results:

Intersection	Traffic volume	Result
int_001	25,000	true
int_002	15,000	false
int_003	32,000	true

Also test an intersection with no traffic-volume fact. It should not be classified as high exposure.

Your turn: Implement `high_traffic_exposure/1` and paste your code here.

I'll review your implementation, explain any mistakes, and then we'll move to pedestrian crossing conditions.

You said:

Please output the walkthrough for steps 2,3 and 4

ChatGPT said:

We'll continue from Task 1 (`high_traffic_exposure/1`). Below are the walkthroughs for Tasks 2, 3 and 4.

We'll keep each task separate so you can implement and test them individually.

Task 2 – Pedestrian crossing conditions

Infrastructure reasoning

Objective: Identify whether an intersection has a pedestrian crossing.

Assume your Python-generated facts look like:

```
pedestrian_crossing(int_001).
pedestrian_crossing(int_003).
```

In this example, `int_002` has no recorded pedestrian crossing.

What you need to implement

Create two predicates:

```
has_pedestrian_crossing(IntersectionID).
no_pedestrian_crossing(IntersectionID).
```

Their expected behaviour:

- `has_pedestrian_crossing/1` succeeds when a pedestrian crossing fact exists.
- `no_pedestrian_crossing/1` succeeds when the intersection exists but has no pedestrian crossing fact.

Hints:

- Use `pedestrian_crossing/1` .
- Use `intersection/1` to ensure the intersection exists.
- Use `\+` for negation.

Testing

```
?- has_pedestrian_crossing(int_001).
?- no_pedestrian_crossing(int_002).
?- no_pedestrian_crossing(int_003).
```

Expected results:

Intersection	Expected
int_001	Has crossing: true
int_002	No crossing: true
int_003	No crossing: false

Your task: Implement both predicates and test them before moving on.

Task 3 – School crossing conditions

Infrastructure reasoning

Objective: Identify intersections with and without school crossings.

Assume these input facts:

```
school_crossing(int_001).
school_crossing(int_002).
```

Here, `int_003` has no recorded school crossing.

What you need to implement

Create:

```
has_school_crossing(IntersectionID).
no_school_crossing(IntersectionID).
```

Their behaviour should mirror your pedestrian crossing predicates.

Hints:

- Use `school_crossing/1`.
- Use `intersection/1`.
- Use `\+` for negation.

Testing

```
?- has_school_crossing(int_001).
?- no_school_crossing(int_003).
?- no_school_crossing(int_002).
```

Expected results:

Intersection	Expected
int_001	Has school crossing: true
int_002	No school crossing: false
int_003	No school crossing: true

Your task: Implement and test both predicates.

Task 4 – Combine contextual reasoning

Hybrid AI reasoning

Objective: Combine the machine-learning output with traffic and infrastructure conditions.

This is where your Prolog knowledge base starts demonstrating the hybrid AI approach.

We want to identify intersections that meet all three conditions:

1. High predicted crash risk.
2. High traffic exposure.
3. No recorded pedestrian crossing.

For this example, assume:

```
predicted_risk(int_001, 0.85).
predicted_risk(int_002, 0.45).
predicted_risk(int_003, 0.90).
```

Let's use `0.7` as the example high-risk threshold.

What you need to implement

First, create:

```
high_predicted_risk(IntersectionID).
```

It should succeed when the predicted risk score is greater than `0.7`.

Then create:

```
high_priority_intersection(IntersectionID).
```

This should succeed when all three conditions are satisfied:

- High predicted risk.
- High traffic exposure.
- No pedestrian crossing.

Hints

Prolog uses commas to represent logical AND.

For example, the general structure of a rule combining three conditions is:

```
some_rule(X) :-
    condition_one(X),
    condition_two(X),
    condition_three(X).
```

You should use your previously implemented predicates rather than repeating their logic.

Testing

Use the following example:

Intersection	Risk	Traffic	Pedestrian crossing	Expected
int_001	0.85	25,000	No	High priority
int_002	0.45	30,000	No	Not high priority
int_003	0.90	32,000	Yes	Not high priority

Test:

```
?- high_predicted_risk(int_001).
?- high_priority_intersection(int_001).
?- high_priority_intersection(int_002).
?- high_priority_intersection(int_003).
```

Expected high-priority result:

```
int_001
```

Important design consideration

For your assessment, distinguish between:

- **No crossing recorded:** There is no crossing fact in the available dataset.
- **Confirmed crossing absence:** The dataset explicitly confirms that no crossing exists.

These aren't necessarily the same thing. Missing infrastructure data should not automatically be interpreted as missing infrastructure.

Also, school crossings should eventually influence your reasoning independently, rather than assuming that a pedestrian crossing and school crossing are interchangeable.

Your implementation checklist

Progress 0/9

- [ ] Implement has_pedestrian_crossing/1
- [ ] Implement no_pedestrian_crossing/1
- [ ] Test Task 2
- [ ] Implement has_school_crossing/1
- [ ] Implement no_school_crossing/1

- Test Task 3
- Implement high_predicted_risk/1
- Implement high_priority_intersection/1
- Test Task 4

My recommendation: Start with Task 2 and send me your two predicates. I'll review your code and test logic before you move to Task 3.

You said:

Help me integrate the Python machine-learning classification into my Prolog knowledge base.
The ML output identifies historical_fsi_present or historical_fsi_absent.
I want Prolog to combine this classification with contextual conditions and data quality to produce:

- priority_safety_review
- safety_review
- contextual_review
- routine_monitoring
- data_review

Guide me through implementing the rules without writing the complete solution for me.

Output one response only.

ChatGPT said:

We'll build this as the final reasoning layer in `crash_risk_rules.pl`.

The key idea is that **Python classifies the historical crash outcome, while Prolog combines that classification with traffic exposure, infrastructure conditions and data quality to determine the recommended review category.**

We'll implement this incrementally, without replacing your existing code.

1. Proposed reasoning structure

Your five output classifications will be:

Prolog classification	Meaning
`priority_safety_review`	Strong evidence requiring priority safety review
`safety_review`	Significant safety concerns requiring review
`contextual_review`	Contextual conditions warrant further investigation
`routine_monitoring`	No significant concerns identified
`data_review`	Insufficient or unreliable information

These are decision-support categories, not definitive safety ratings.

2. Suggested input facts

Python supplies the historical FSI classification:

```
historical_fsi_present(int_001).
historical_fsi_absent(int_002).
```

Your existing Prolog knowledge base supplies contextual information:

```
high_traffic_exposure(int_001).
no_pedestrian_crossing(int_001).
has_school_crossing(int_001).
```

And your data-quality predicates identify problems:

```
missing_traffic_volume(int_002).
stale_traffic_volume(int_003).
incomplete_intersection(int_004).
```

Use your actual predicate names if they differ.

3. Implementation plan

We'll divide the implementation into five tasks.

Task	What you'll implement
1	Validate ML classification availability
2	Create contextual concern predicates
3	Establish data-quality precedence
4	Combine conditions into review categories
5	Test conflicting conditions and classifications

Task 1 – Validate ML classification availability

First coding task

Before making decisions, Prolog needs to know whether Python supplied a valid classification.

Your objective: Create a predicate:

```
valid_ml_classification(IntersectionID).
```

It should succeed when an intersection has either:

- `historical_fsi_present/1`
- `historical_fsi_absent/1`

It should fail if neither classification exists.

It should also avoid treating an intersection with both classifications as valid.

Hints:

You can use:

- `historical_fsi_present/1`
- `historical_fsi_absent/1`
- `\+` for negation
- `,` for AND
- `;` for OR

Think about how you could check that exactly one classification exists.

Test cases:

```
historical_fsi_present(int_001).
historical_fsi_absent(int_002).
```

Expected behaviour:

Intersection	Expected
int_001	true
int_002	true
int_003 (no classification)	false
int_004 (both classifications)	false

Task 2 – Create contextual concern predicates

Once Task 1 works, we'll establish reusable contextual conditions.

Suggested predicates:

```
high_risk_context(IntersectionID).
moderate_risk_context(IntersectionID).
low_risk_context(IntersectionID).
```

Possible reasoning:

High-risk context

- High traffic exposure.
- No recorded pedestrian crossing.

Moderate-risk context

- High traffic exposure.
- Pedestrian crossing present, but school crossing conditions may require further consideration.

Low-risk context

- Traffic exposure is not high.
- No significant infrastructure concerns identified.

Don't implement these yet. We'll decide the exact conditions and how to handle unknown information when we reach Task 2.

Task 3 – Establish data-quality precedence

This is particularly important.

Data-quality problems should take precedence over ordinary safety classifications.

For example:

```
Missing traffic volume
        |
        v
Data review
```

Potential conditions include:

- Missing traffic-volume information.
- Stale traffic-volume information.
- Incomplete intersection information.
- Missing ML classification.
- Conflicting ML classifications.

The important principle is that Prolog shouldn't silently interpret missing information as evidence of safety.

Task 4 – Combine everything into the five outputs

Your final classification layer will have these conceptual relationships:



We'll need to define explicit precedence so an intersection cannot accidentally receive multiple conflicting classifications.

For example, a possible initial policy is:

1. Data quality failure → `data_review`.
2. Historical FSI present + high-risk context → `priority_safety_review`.
3. Historical FSI present → `safety_review`.
4. Historical FSI absent + concerning context → `contextual_review`.
5. Historical FSI absent + no significant contextual concerns → `routine_monitoring`.

This is a proposed policy, not a claim that historical FSI absence means an intersection is safe. Historical crash records cannot establish future safety.

Task 5 – Test the final classifications

Eventually, test at least these scenarios:

Scenario	Expected output
FSI present + high traffic + no pedestrian crossing	Priority safety review
FSI present + lower contextual risk	Safety review
FSI absent + high traffic + no pedestrian crossing	Contextual review
FSI absent + no significant concerns	Routine monitoring
Missing or stale required data	Data review
Conflicting ML classifications	Data review

4. Your first implementation task

Let's start with `valid_ml_classification/1`.

Write the predicate yourself and test the four cases from Task 1.

One important question: Does Python generate exactly one of `historical_fsi_present/1` or `historical_fsi_absent/1` for every intersection, or can it sometimes generate neither?

This will help us establish the correct data-quality handling.

Paste your implementation here when ready. I'll review it before we move to contextual reasoning.

You said:

Help me implement explanatory predicates in crash_risk_rules.pl so the system can explain why an assessment was generated.

Then help me test the rules using positive and negative ML classifications, contextual concerns and missing or stale data.

Output all tasks and help me review if there are any major mistakes.

ChatGPT said:

We'll add an **explanation layer** to `crash_risk_rules.pl` so that every assessment can provide understandable reasons for its classification.

This is particularly important for your COMP3742/9742 hybrid AI project: Python provides the ML classification, while Prolog provides transparent, traceable reasoning.

We'll use a structured approach:

1. Create explanatory predicates.
2. Connect explanations to the five assessment categories.
3. Test positive and negative ML classifications.
4. Test contextual conditions.
5. Test missing, stale and conflicting data.
6. Review the knowledge base for major logical mistakes.

I'll give you the tasks and expected results, but leave the implementation to you.

1. Recommended knowledge base structure

Keep your existing facts and rules. Organise the file into these sections:

```
1. Input facts (Python-generated)
2. Data availability predicates
3. Data quality predicates
4. Traffic exposure predicates
5. Pedestrian and school crossing predicates
6. Contextual reasoning predicates
7. ML classification validation
8. Final assessment predicates
9. Explanation predicates
```

The explanation section should come after the assessment rules because it needs to inspect their results.

2. Explanation predicates

We want Prolog to answer questions such as:

- Why was this intersection prioritised?
- Was historical FSI recorded?
- Was traffic exposure high?

- Was pedestrian infrastructure recorded?
- Was the data incomplete or stale?

I recommend using a predicate that returns a list of explanation reasons.

For example, your eventual query could look like:

```
?- explain_assessment(int_001, Reasons).
```

Expected conceptual output:

```
Reasons = [
    historical_fsi_present,
    high_traffic_exposure,
    no_pedestrian_crossing,
    priority_safety_review
].
```

These are illustrative reasons, not yet an implementation.

Task 1 – Explain the ML classification

Objective: Create a predicate that returns the ML classification as an explanation.

Implement:

```
explain_ml_classification(IntersectionID, Reason).
```

It should return one of:

- `historical_fsi_present`
- `historical_fsi_absent`
- `ml_classification_missing`
- `ml_classification_conflict`

Hints:

- Reuse `historical_fsi_present/1`.
- Reuse `historical_fsi_absent/1`.
- Reuse your `valid_ml_classification/1`.
- Check for both classifications before checking for just one.

Testing:

Input facts	Expected explanation
FSI present only	`historical_fsi_present`
FSI absent only	`historical_fsi_absent`
Neither	`ml_classification_missing`
Both	`ml_classification_conflict`

Your predicate should not silently choose one classification when both exist.

Task 2 – Explain contextual conditions

Objective: Return reasons based on traffic and infrastructure conditions.

Implement:

```
explain_context(IntersectionID, Reason).
```

Your predicate should identify these conditions:

Condition	Explanation returned
Traffic above 20,000	`high_traffic_exposure`
Traffic at or below 20,000	`lower_traffic_exposure`
Pedestrian crossing recorded	`pedestrian_crossing_present`
No pedestrian crossing recorded	`pedestrian_crossing_not_recorded`

School crossing recorded	`school_crossing_present`
No school crossing recorded	`school_crossing_not_recorded`

Important: Use `not_recorded` rather than `absent` unless your dataset explicitly confirms the infrastructure is absent.

Hints:

- Reuse your existing predicates.
- You can have multiple clauses with the same predicate name.
- Each successful query should return one explanation.
- Avoid treating missing traffic-volume data as lower traffic exposure.

Testing:

Use an intersection with:

- 25,000 vehicles.
- No recorded pedestrian crossing.
- A school crossing.

Expected explanations:

```
high_traffic_exposure
pedestrian_crossing_not_recorded
school_crossing_present
```

Also test an intersection with missing traffic-volume information. It should not return `lower_traffic_exposure`.

Task 3 – Explain data-quality problems

Objective: Make Prolog explain why an intersection requires a data review.

Implement:

```
explain_data_quality(IntersectionID, Reason).
```

Potential explanations:

Condition	Explanation
Missing traffic volume	`missing_traffic_volume`
Stale traffic volume	`stale_traffic_volume`
Incomplete intersection	`incomplete_intersection`
Missing ML classification	`ml_classification_missing`
Conflicting ML classifications	`ml_classification_conflict`

Hints:

Reuse your existing data-quality predicates rather than duplicating their logic.

One intersection may have multiple data-quality problems. Your predicate should be capable of returning each applicable reason.

Testing:

Create an intersection with:

- Missing traffic volume.
- Missing ML classification.
- Incomplete intersection information.

Expected results should include all three relevant explanations.

3. Connect explanations to final assessments

Task 4 – Explain the final assessment

Now create:

```
explain_assessment(IntersectionID, Reasons).
```

This should collect the explanations from your previous tasks and include the final assessment category.

For example:

```
?- explain_assessment(int_001, Reasons).
```

Conceptual output:

```
Reasons = [
    historical_fsi_present,
    high_traffic_exposure,
    pedestrian_crossing_not_recorded,
    priority_safety_review
].
```

For an intersection requiring data review, the explanation might look like:

```
Reasons = [
    missing_traffic_volume,
    stale_traffic_volume,
    data_review
].
```

Hints:

- Use `findall/3` to collect multiple explanation reasons.
- Use your existing assessment predicate.
- Add the final assessment category to the explanation list.
- Avoid adding duplicate reasons.

A useful Prolog concept to understand here is:

```
findall(Template, Goal, List).
```

It finds every solution to `Goal` and places the corresponding `Template` values into `List`.

For this task, you can use it to collect all applicable explanations.

Task 5 – Ensure every intersection receives one assessment

Implement or review:

```
intersection_assessment(IntersectionID, Assessment).
```

It should return exactly one of:

```
priority_safety_review
safety_review
contextual_review
routine_monitoring
data_review
```

Your assessment rules need a clear precedence order.

I recommend:

Priority	Condition	Assessment
1	Missing, stale, incomplete or conflicting required data	`data_review`
2	FSI present + high traffic + pedestrian crossing not recorded	`priority_safety_review`
3	FSI present, without meeting priority conditions	`safety_review`
4	FSI absent + concerning contextual conditions	`contextual_review`
5	FSI absent + no significant concerns	`routine_monitoring`

This is an initial decision policy. Your team should document and justify the thresholds and conditions.

Important: An FSI-absent classification means no historical FSI was identified in the relevant data. It does not establish that the intersection is safe.

4. Testing the complete knowledge base

We'll use a controlled test dataset to check that your rules behave correctly.

Task 6 – Positive and negative ML classifications

Create test intersections with the following classifications:

Intersection	ML classification
int_001	FSI present
int_002	FSI absent
int_003	FSI present
int_004	FSI absent
int_005	Missing classification
int_006	Both classifications

Test:

```
?- valid_ml_classification(int_001).
?- valid_ml_classification(int_002).
?- valid_ml_classification(int_005).
?- valid_ml_classification(int_006).
```

Expected results:

Intersection	Expected
int_001	true
int_002	true
int_005	false
int_006	false

Task 7 – Test contextual conditions

Use these scenarios:

Intersection	Traffic	Pedestrian crossing	School crossing
int_001	25,000	No	No
int_002	15,000	Yes	No
int_003	30,000	Yes	Yes
int_004	Missing	Unknown	Unknown

Test your existing predicates:

```
?- high_traffic_exposure(int_001).
?- has_pedestrian_crossing(int_002).
?- has_school_crossing(int_003).
?- high_traffic_exposure(int_004).
```

Expected results:

- `int_001` : high traffic.
- `int_002` : pedestrian crossing present.
- `int_003` : school and pedestrian crossings present.
- `int_004` : no high-traffic classification.

Task 8 – Test missing and stale data

Test these cases independently:

Intersection	Traffic volume	Collection year	Expected
int_007	Missing	Missing	Missing data
int_008	25,000	2020	Stale data
int_009	25,000	2025	Valid data under the example policy

int_010	15,000	2025	Valid data under the example policy

For this test, continue using your previous assumptions:

- Assessment year: 2026.
- Data older than three years: stale.
- High traffic: greater than 20,000 vehicles.

Make sure that stale data does not get treated as valid high traffic exposure.

Task 9 – Test final assessment outcomes

Your final integration test should cover all five categories.

Intersection	ML classification	Context	Data quality	Expected
int_001	Present	High traffic, no pedestrian crossing	Valid	Priority safety review
int_002	Present	Lower traffic, crossing present	Valid	Safety review
int_003	Absent	High traffic, no pedestrian crossing	Valid	Contextual review
int_004	Absent	Lower traffic, crossings recorded	Valid	Routine monitoring
int_005	Present	High traffic	Missing volume	Data review
int_006	Absent	Lower traffic	Stale volume	Data review

These are policy test cases, not claims about actual road safety.

5. Major mistakes to watch for

These are the issues I'd pay particular attention to when reviewing your implementation.

⚠ 1. Negation and missing data

Using `\+ pedestrian_crossing(X)` does not prove that a crossing is absent. It only means no matching fact was found.

Distinguish unknown information from confirmed absence.

⚠ 2. Multiple assessment categories

Prolog can return multiple solutions if several assessment rules match.

Make sure your final classification has explicit precedence and returns exactly one result.

⚠ 3. Stale data

Don't allow stale traffic-volume information to contribute to a valid exposure classification.

⚠ 4. ML classification conflicts

An intersection must not be considered valid if both `historical_fsi_present/1` and `historical_fsi_absent/1` exist.

⚠ 5. Duplicate explanation reasons

Be careful when combining explanation lists. The same reason might be generated by multiple predicates.

⚠ 6. Unbound variables

Ensure your predicates receive a known intersection ID before using negation or arithmetic comparisons.

6. Final verification checklist

Before considering `crash_risk_rules.pl` complete, check:

- ○ Python-generated facts load successfully into SWI-Prolog.
- ○ Every intersection has at most one valid ML classification.
- ○ Missing traffic-volume information is detected.
- ○ Stale traffic-volume information is detected.
- ○ Traffic threshold is exactly 20,000, with high exposure defined as greater than 20,000.
- ○ Pedestrian and school crossing conditions are evaluated independently.
- ○ Data-quality problems take precedence over safety classifications.
- ○ Exactly one final assessment is returned per intersection.
- ○ Explanation predicates return all relevant reasons.

- ◯ Missing information is not incorrectly treated as confirmed infrastructure absence.
- ◯ All five assessment categories have passing tests.

One additional recommendation: Keep a separate test file, such as `crash_risk_tests.pl`, rather than filling your production knowledge base with test facts. SWI-Prolog's `plunit` testing framework is useful for this.

7. How we'll review your implementation

When you're ready, send me your `crash_risk_rules.pl` file or paste its contents.

I'll review it in four areas:

Review area	What I'll check
Syntax	SWI-Prolog errors, predicate definitions
Logic	AND/OR conditions, negation, precedence
Integration	Python facts and ML classifications
Explainability	Whether assessment reasons accurately reflect the rules

I'll identify major mistakes first, explain why they matter, and give you small corrections rather than rewriting the file.

My recommendation: Implement Tasks 1–3 first, then Tasks 4–5. Test each stage before moving forward. This will make debugging considerably easier and help you explain the reasoning architecture during your assessment.

You said:

crash_risk_rules.pl
Code

I have completed most of crash_risk_rules.pl.

Please help me check its integration with prolog_interface.py and the other Python components.

Review predicate names, arguments, returned assessment outcomes and error handling.

Identify problems and explain how I can fix them myself. Do not rewrite existing code.

Worked for 28s

ChatGPT said:

I've reviewed your uploaded `crash_risk_rules.pl`. The overall structure is sensible: you've separated data validation, contextual reasoning, ML validation and final assessment rules.

You've also made a good design choice by keeping `site_assessment/3` as the main decision-making predicate and using `recommendation/3` as a compatibility wrapper.

However, I've identified several integration and logic issues worth fixing before testing it with Python.

Important: I only have `crash_risk_rules.pl` in this conversation. I haven't yet verified the actual Python-to-Prolog interface or the generated fact files, so I'll distinguish confirmed Prolog issues from integration questions.

1. Predicate interface review

Your Prolog knowledge base currently expects the following predicates and arguments:

Predicate	Arguments	Purpose
`site/1`	Site	Identifies an intersection
`traffic_volume/2`	Site, Volume	Traffic volume
`has_volume/1`	Site	Availability flag
`volume_current/1`	Site	Freshness flag
`volume_stale/1`	Site	Stale-data flag
`pedestrian_crossings/2`	Site, Count	Pedestrian crossing count
`school_crossings/2`	Site, Count	School crossing count
`ml_classification/2`	Site, Class	ML classification
`ml_positive_probability/2`	Site, Probability	ML probability

These names and argument structures must match the Python-generated Prolog facts exactly.

For example, Python generating:

```
historical_fsi_present(int_001).
```

would not match your current expectation:

```
ml_classification(int_001, historical_fsi_present).
```

This is an important integration detail.

Your ML interface also expects a probability between 0 and 1, not a percentage between 0 and 100. crash_risk_rules.pl

2. Major problems identified

Needs attention

Problem 1 – Incorrect final assessment name

Your required output categories are:

- `priority_safety_review`
- `safety_review`
- `contextual_review`
- `routine_monitoring`
- `data_review`

However, your Rule 7 currently returns:

```
routine_review
```

This is inconsistent with your required Python output categories.

How to fix it: Find Rule 7 around lines 426–434 and change the returned assessment atom to match your agreed interface.

crash_risk_rules.pl

Problem 2 – Conflicting ML classifications aren't detected

Your `ml_prediction_available/1` checks whether a valid classification exists.

However, it doesn't ensure that exactly one classification exists.

For example:

```
ml_classification(int_001, historical_fsi_present).
ml_classification(int_001, historical_fsi_absent).
```

Both `ml_positive_class/1` and `ml_negative_class/1` can succeed.

This is a significant problem because your assessment could become dependent on which rule is evaluated first.

How to fix it yourself:

Review the logic around lines 236–261.

You need to establish that:

- A valid positive classification exists.
- A valid negative classification exists.
- Both must not exist simultaneously.
- A missing classification is handled according to your chosen data-quality policy.

crash_risk_rules.pl

Problem 3 – Pedestrian crossing presence is treated as a concern

Your current contextual rules include:

```
contextual_concern(Site) :-
    pedestrian_crossing_present(Site).

contextual_concern(Site) :-
    school_crossing_present(Site).
```

This means that having a pedestrian crossing or school crossing can trigger a higher-priority assessment.

That might not reflect your intended safety reasoning.

How to fix it yourself:

Consider separating:

- Infrastructure availability.
- Infrastructure gaps.
- Contextual risk indicators.

A recorded crossing is infrastructure availability. It is not automatically evidence of increased risk.

You might eventually want a separate condition for intersections with high traffic and no recorded pedestrian crossing.

crash_risk_rules.pl

Problem 4 – Missing ML output has inconsistent handling

There is an important policy difference between your comments and your assessment rules.

Your code deliberately allows an intersection with completely missing ML output to receive a contextual or routine assessment.

For example, Rule 6 and Rule 7 contain:

```
\+ ml_prediction_available(Site),
\+ ml_output_issue(Site)
```

This means missing ML output is not necessarily a `data_review`.

However, your earlier design treated missing ML classification as a data-quality problem.

How to fix it yourself:

Decide which behaviour your team wants:

- **Strict validation:** Missing ML output results in `data_review`.
- **Contextual fallback:** Allow Prolog to assess the available contextual information without ML output.

For a hybrid AI assessment, I'd favour strict validation if the ML classification is a required input to the final assessment. If you retain fallback behaviour, make it an explicitly documented exception.

crash_risk_rules.pl

Problem 5 – Your crossing data affects assessment eligibility

Your `usable_site/1` predicate calls:

```
\+ data_issue(Site, _).
```

But your crossing validation rules classify missing pedestrian and school crossing information as data issues.

Consequently, an intersection with valid traffic and ML data but missing crossing information cannot proceed through the normal assessment rules.

That might be intentional, but it is quite strict.

How to fix it yourself:

Consider whether missing crossing information should:

- Prevent assessment entirely; or
- Allow assessment with an uncertainty explanation.

I recommend the second approach if you want your system to make useful decisions when some infrastructure datasets are incomplete.

crash_risk_rules.pl crash_risk_rules.pl

Problem 6 – Traffic threshold boundary

Your current rule uses:

```
Volume >= Threshold
```

Your earlier requirement described high traffic as greater than 20,000 vehicles.

These differ when traffic volume is exactly 20,000.

How to fix it yourself: Decide whether 20,000 belongs in the high-exposure category, then ensure the Python and Prolog implementations use the same comparison.

crash_risk_rules.pl

Problem 7 – Only the first data issue becomes the main explanation

Your `site_data_issues/2` predicate correctly collects all recognised data issues using `findall/3` and `sort/2`.

However, `site_assessment/3` uses a cut (`!`) after finding the first issue.

This means the main assessment explanation only contains one issue, even when several exist.

This isn't necessarily incorrect: your code comments say all issues can be retrieved separately.

How to fix it yourself: Decide whether Python needs:

- The primary assessment and one explanation; or
- The primary assessment and the complete list of data-quality issues.

The second option would provide better explainability.

crash_risk_rules.pl crash_risk_rules.pl

Problem 8 – Undefined predicates and error handling

Your rule file calls several predicates that may not have any facts:

```
has_volume/1
volume_current/1
pedestrian_crossings/2
school_crossings/2
ml_classification/2
ml_positive_probability/2
```

If the generated files don't define these predicates at all, SWI-Prolog can raise an `existence_error` when they are called.

This is different from a predicate existing but having no matching facts.

How to fix it yourself:

Check whether your generated `.pl` files:

- Define all expected predicates.
- Declare predicates that may have no facts.
- Use consistent argument types.
- Handle empty datasets.

Also inspect your loading predicates:

```
load_site_facts(FactsFile) :-
    consult(FactsFile).

load_ml_facts(FactsFile) :-
    consult(FactsFile).
```

Currently, there is no explicit handling for missing files or failed consultations.

Consider how your Python interface will detect and report a Prolog loading failure rather than continuing with incomplete facts.

crash_risk_rules.pl

3. Python integration: files I need to inspect

I can verify the Prolog side, but I cannot honestly confirm compatibility with your Python implementation without seeing its source.

Please upload these files:

Required files

‹/› `prolog_interface.py` — Required
Most important – queries, loading and returned results.

‹/› **ML fact-generation Python file** — Required
Where classification and probability facts are created.

Site fact-generation Python file
Where traffic and crossing facts are generated.
Helpful

Pipeline entry point or integration tests
How the components are executed together.
Helpful

4. What I'll check once you upload them

I'll perform an integration review without rewriting your existing code.

Area	Specific checks
Predicate compatibility	Names, argument counts and data types
Fact generation	Correct Prolog syntax and escaping
ML integration	FSI classification and probability format
Assessment results	All five required output categories
Data quality	Missing, stale and invalid values
SWI-Prolog execution	File paths, loading and query execution
Error handling	Missing files, Prolog exceptions and failed queries
Result parsing	Correct extraction of assessment and explanation

I'll also check whether your Python interface expects `site_assessment/3`, `recommendation/3`, or another predicate.

5. Current priority order

Before integrating everything, I'd address these issues in this order:

1. Correct `routine_review` to `routine_monitoring`.
2. Detect conflicting ML classifications.
3. Resolve the missing-ML-output policy.
4. Reconsider whether crossing presence should trigger contextual concern.
5. Decide how incomplete crossing information affects eligibility.
6. Verify predicate declarations and loading errors.
7. Align the Python and Prolog interfaces.

My overall assessment: Your Prolog file has a good foundation, but I would not consider the hybrid system integration-ready until the classification conflicts, contextual reasoning and missing-data behaviour are resolved.

Once you upload the Python files, I'll trace the data from Python fact generation through SWI-Prolog execution to the final returned assessment, identifying exactly where the interfaces match or break.